LLM and Software Security

Jingyue Li

13 March 2026

Agenda

AI-assisted software development

Benefits

Risks

AI-assisted malware detection

Agent-based vulnerability fixing recommendation

Your experience in using AI-assisted vulnerability detection

AI-assisted software development - Benefit

FROM MANUAL CODING TO AI COLLABORATION

Tech progression: Text editors → IDEs → Autocompletion

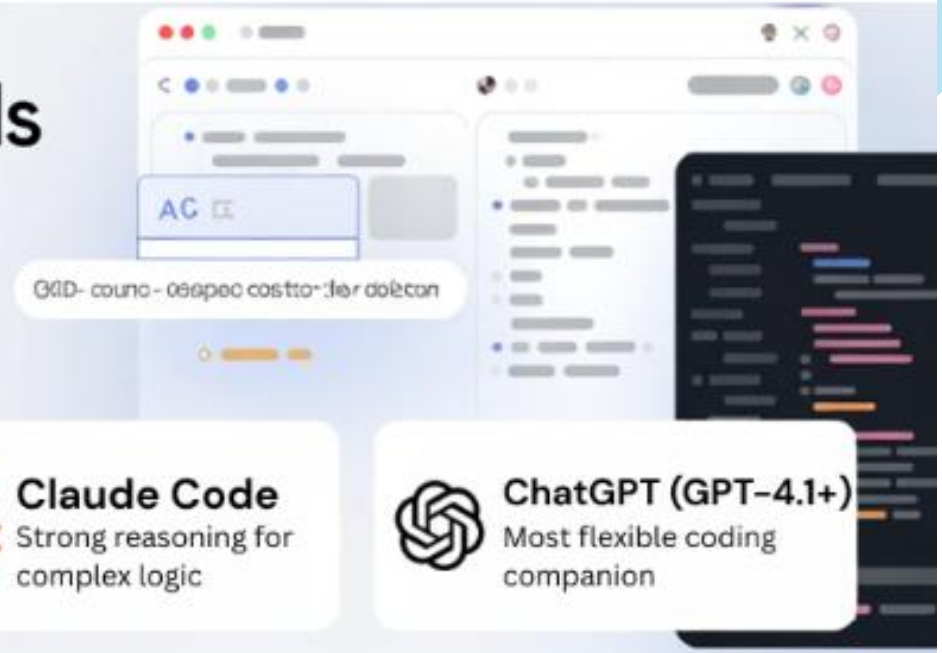
→ AI assistance

- Shift from syntax help to semantic understanding
- From isolated editing to continuous collaboration

WHAT AI CODING ASSISTANTS CAN DO?

- Automate repetitive tasks
- Reduce cognitive load
- Accelerate navigation of unfamiliar languages/frameworks
- Maintain "flow state" during development
- Democratize coding expertise

10 Best AI Coding Assistant tools in 2026 (Developer-tested)



GitHub Copilot
Best for everyday professional coding

Cursor
AI-first IDE for large codebases

Claude Code
Strong reasoning for complex logic

ChatGPT (GPT-4.1+)
Most flexible coding companion

Amazon Q Developer
Ideal for AWS-based projects

Tabnine
Privacy-focused AI for teams

Replit Ghostwriter
Fast browser-based development

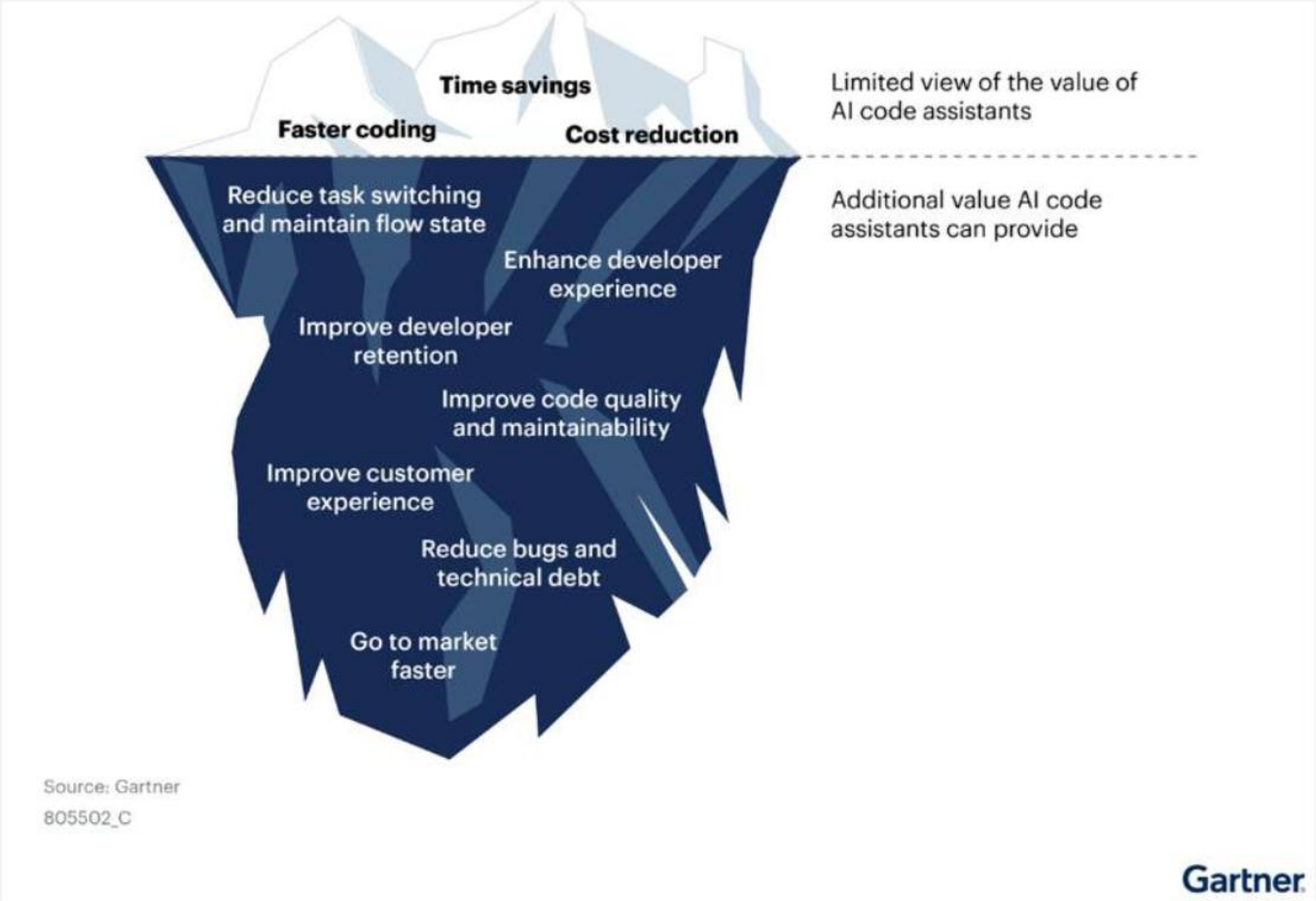
Codeium
Free AI assistant with wide IDE support

CodeGPT
Customizable AI for multiple models

Continue.dev
Open-source AI coding agent

Tech Jarvis Ai
The Source Code for AI Knowledge

VALUE OF AI CODE ASSISTANTS



https://www.gartner.com/en/newsroom/press-releases/2024-04-11-gartner-says-75-percent-of-enterprise-software-engineers-will-use-ai-code-assistants-by-2028

IMPROVING DEVELOPER EXPERIENCE

FOCUS ON WHAT MATTERS MOST

- Designing
- Brainstorming
- Collaborating
- Iterating
- Planning

LESS TIME ON

- Writing Tests, Repetitive Code, & Boilerplate
- Debugging
- Searching Documentation
- Manually Finding Vulnerabilities
- Deciphering Existing Code
- Correcting Syntax
- Summarizing Changes and Comments
- Learning Git Commands

AI-assisted software development - Risks

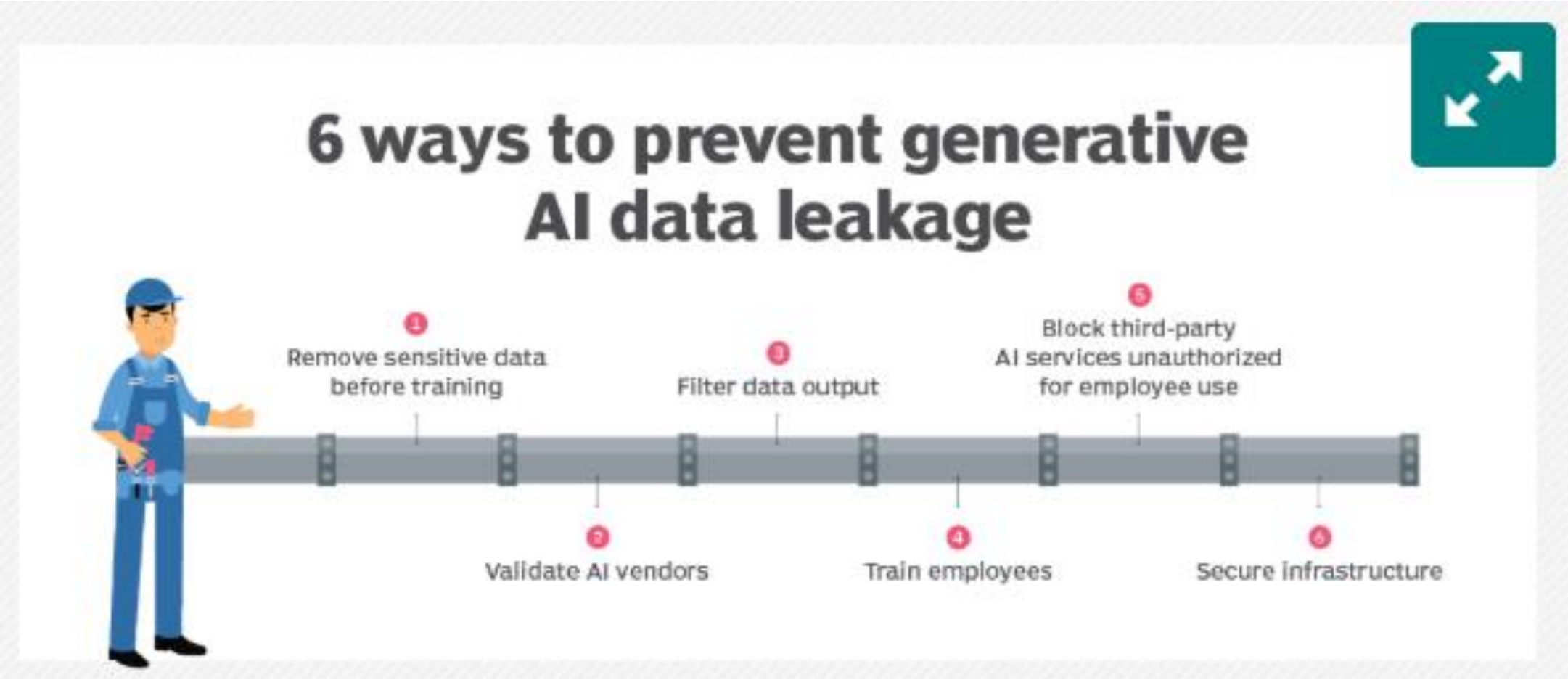
POTENTIAL RISK CATEGORIES

Sensitive data leaks

Suggesting vulnerable code

Overlooking security

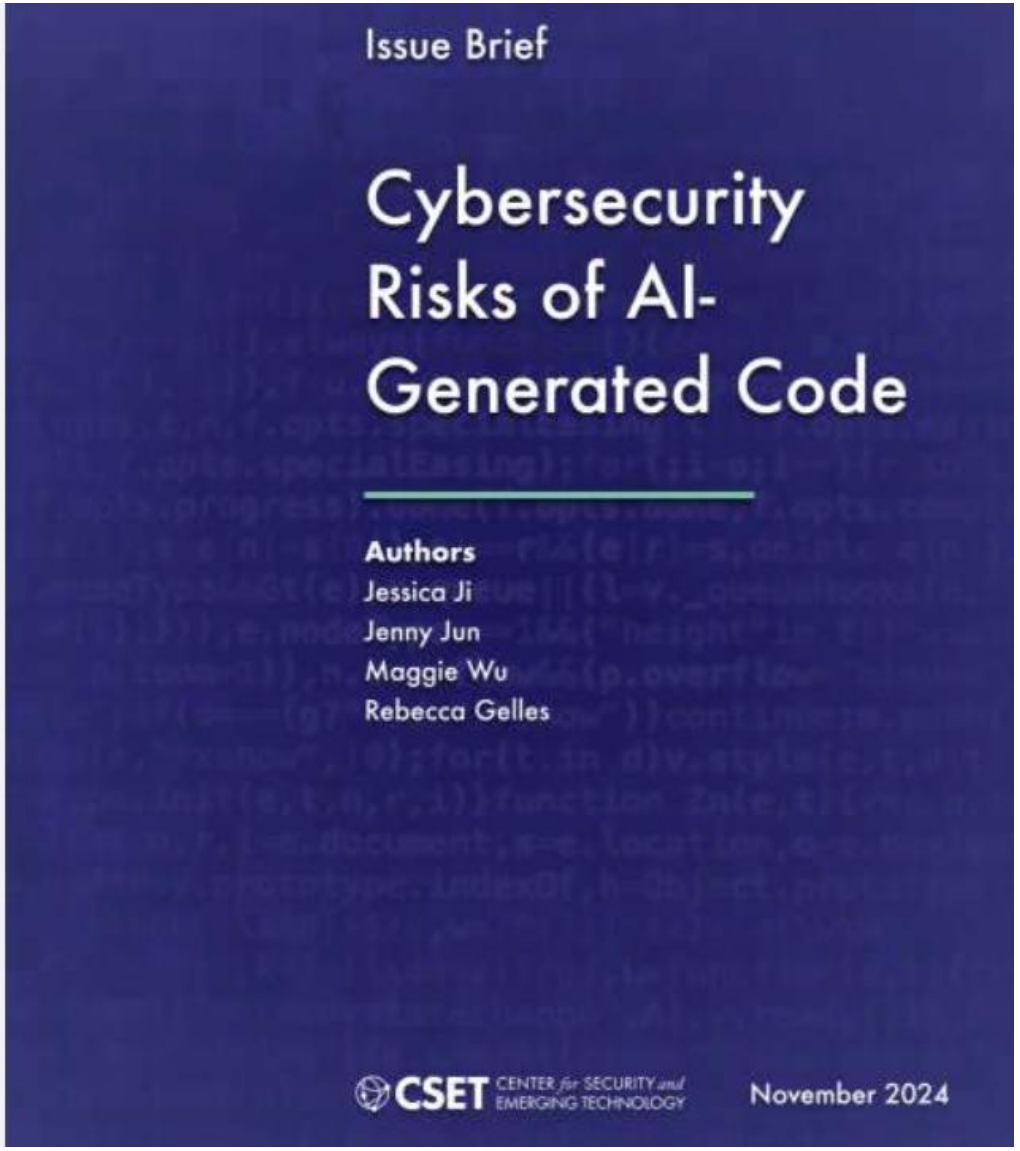
Sensitive data leaks



https://www.techtarget.com/searchenterpriseai/answer/How-bad-is-generative-AI-data-leakage-and-how-can-you-stop-it

48%

of the code produced by five different LLMs contains at least one bug that could potentially lead to malicious exploitation


Issue Brief

Cybersecurity Risks of AI-Generated Code

Authors
Jessica Ji
Jenny Jun
Maggie Wu
Rebecca Gelles

CSET CENTER for SECURITY and EMERGING TECHNOLOGY

November 2024

GARBAGE IN, GARBAGE OUT

- Public repositories (GitHub, BitBucket, etc.)
- Open-source projects
- Stack Overflow and developer forums
- Documentation and code examples

Potential inclusion of vulnerable code patterns

BROAD CATEGORIES OF RISK OF AI CODE GENERATION

Models generating insecure code

Models themselves are vulnerable to attack and manipulation

Downstream cybersecurity impacts such as feedback loops in training future AI systems

MISSING SECURITY CONTEXT

Struggles with unfamiliar data domains

Limited awareness of environmental security requirements

AI CODING ASSISTANTS:

Security or sense of security?

DO USERS WRITE MORE INSECURE CODE WITH AI ASSISTANTS?

Percentage of coders submitting secure answers to coding questions (Using AI vs Not Using AI) *

* Custom-built AI coding assistant based on OpenAI's Codex

https://arxiv.org/pdf/2211.03622, Stanford University

We observed that participants who had access to the AI Assistants were more likely to introduce security vulnerabilities across the majority of programming tasks, yet were also ***more likely to rate their insecure answers as secure*** than those in our control group.

https://arxiv.org/pdf/2211.03622, Stanford University

GOOD DEVELOPER PRACTICES

Choose your AI assistant wisely

Apply secure prompt engineering

Add real-time vulnerability detection tools

Embed security in development workflow

Human-in-the-loop validation

SECURE PROMPT ENGINEERING

- Explicit security requirements in prompts
- Framework-specific security guidance
- Context-setting for security-critical components
- Example-driven prompting with secure patterns

```
Generate a function to authenticate users against a database that follows OWASP secure
coding practices. Ensure password hashing with bcrypt, proper error handling without
information disclosure, and protection against injection attacks.
```

REALTIME VULNERABILITY DETECTION TOOLS

SNYK CODE

DEVELOPER-FOCUSED, REAL-TIME SAST

Secure your code as it's written with static application security testing built by, and for, developers.

SonarQube ide

Features

IDE EXTENSION. SONARQUBE FOR IDE. MORE THAN A LINTER.

An advanced linter in your IDE for Clean Code

AI code remediation

Veracode Fix

Give developers the AI tools they need to fix security flaws in minutes.

AI-assisted malware detection

Eduard Andrei Cristea, Petter Molnes, Jingyue Li: MalCVE: Malware Detection and CVE Association Using Large Language Models, Accepted by the 3rd ACM International Conference on AI Foundation Models and Software Engineering (FORGE 2026), https://arxiv.org/abs/2510.15567

Challenges

Malware costs are projected to be over 276 billion USD per year by 2035

Commercial malware detection tools are expensive, e.g.,

ANY.RUN:
$2.40 per file

CrowdStrike:
$2 per file

Research question

- Can LLMs accurately classify decompiled binaries as either malicious or benign?

Steps

Decompiling the JAR file

Deobfuscating the code

Summarizing the code using LLM

Step 1: Decompiling the JAR file

- We used open source tools (limited to Java)
  - CFR 0.152
  - Procyon 0.6.0 (if CFR does not work)

Step 2: Deobfuscating the code

- Open source tools, e.g., Java Deobfuscator, could not work
- We developed our own Deobfuscator

```
4 public class rxkxU {
5
6     public static Object jWu(Object object) throws Exception {
          String string = "getMBeanInsta" + "VVVVnVtViVaVtVoVVVrVVV".replace("V", "");
          System.out.println(string);
          return object.getClass().getMethod(string, new Class[0]).invoke(object, new Object[0]);
7         String string = "getMBeanInstantiator";
8         System.out.println("getMBeanInstantiator");
9         return object.getClass().getMethod("getMBeanInstantiator", new Class[0]).invoke(object, new Object[0]);
10    }
11
12    public static String WboqbLRDAM() {
13        String string = "VVVVcVoVmV.VVVVVsVuVnVVVVVVVVVV.VjVmVx.mbVVVVVeansVeVrVvVeVrV.VMVBVeVaVnVIVnVVsVtVaVnVtViVaVtVoVrV";
          System.out.println(string);
          return string.replace("V", "");
14        System.out.println("VVVVcVoVmV.VVVVVsVuVnVVVVVVVVVV.VjVmVx.mbVVVVVeansVeVrVvVeVrV.VMVBVeVaVnVIVnVVsVtVaVnVtViVaVtVoVrV");
15        return "com.sun.jmx.mbeanserver.MBeanInstantiator";
16    }
17
18    public static String UIKg(String string, String string2) {
19        String[] stringArray = string.split(string2);
20        String string3 = "";
21        System.out.println(stringArray.length);
22        for (int i = 0; i < stringArray.length; ++i) {
              string3 = string3 + (char)(Integer.parseInt(stringArray[i]) + 14);
23            string3 = string3 + (char) (Integer.parseInt(stringArray[i]) + 14);
24        }
          return string3;
25        return "";
26    }
27
28    public static String osCqBi() {
          return "MMMMfMiMMMnM".replace("M", "").concat("MMMMMMMMMdMCM").replace("M", "").concat("MMMMMMMlMaMMMMsMMMMsMMM").replace("M", "");
29        return "findClass";
30    }
31
32    public static String LJreSfqItu() {
          String string = "MMMMjMMMMaMMMMvMMMMMaMMMMM".replace("M", "") + "MMMM.MMMMiMMMMMoMMMMM.MtMaMMMMMMMMMMM".replace("M", "") + "pMMMMMMMMdMiMMMMrMMMMMMMMM".replace("M", "");
          return string;
33        String string = "java.io.tmpdir";
34        return "java.io.tmpdir";
35    }
36 }
```

```java
public static Object jWu(Object object) throws Exception {
    String string = "getMBeanInsta" + "VVVVnVtViVaVtVoVVVrVVV".replace("V", "");
    System.out.println(string);
    return object.getClass().getMethod(string, new Class[0]).invoke(object, new Object[0]);
    String string = "getMBeanInstantiator";
    System.out.println("getMBeanInstantiator");
    return object.getClass().getMethod("getMBeanInstantiator", new Class[0]).invoke(object, new Object[0]);
}
```

- Walking the AST (Abstract Syntax Tree) of the code
- Evaluating constant-like expressions (especially string-related ones)
- Replacing obfuscated expressions in the source with their computed literal values

Step 3: Summarizing the code using LLM

Prompt

Act as a malware analyst and thoroughly examine this decompiled executable code. Break it down step by step, identifying key behaviors, suspicious patterns, and potential vulnerabilities.

1. **Verdict**: Classify the file as Benign, Suspicious, or Malicious, with a confidence score and rationale.
2. **Summary**: Provide a clear explanation of what this code does.

Output

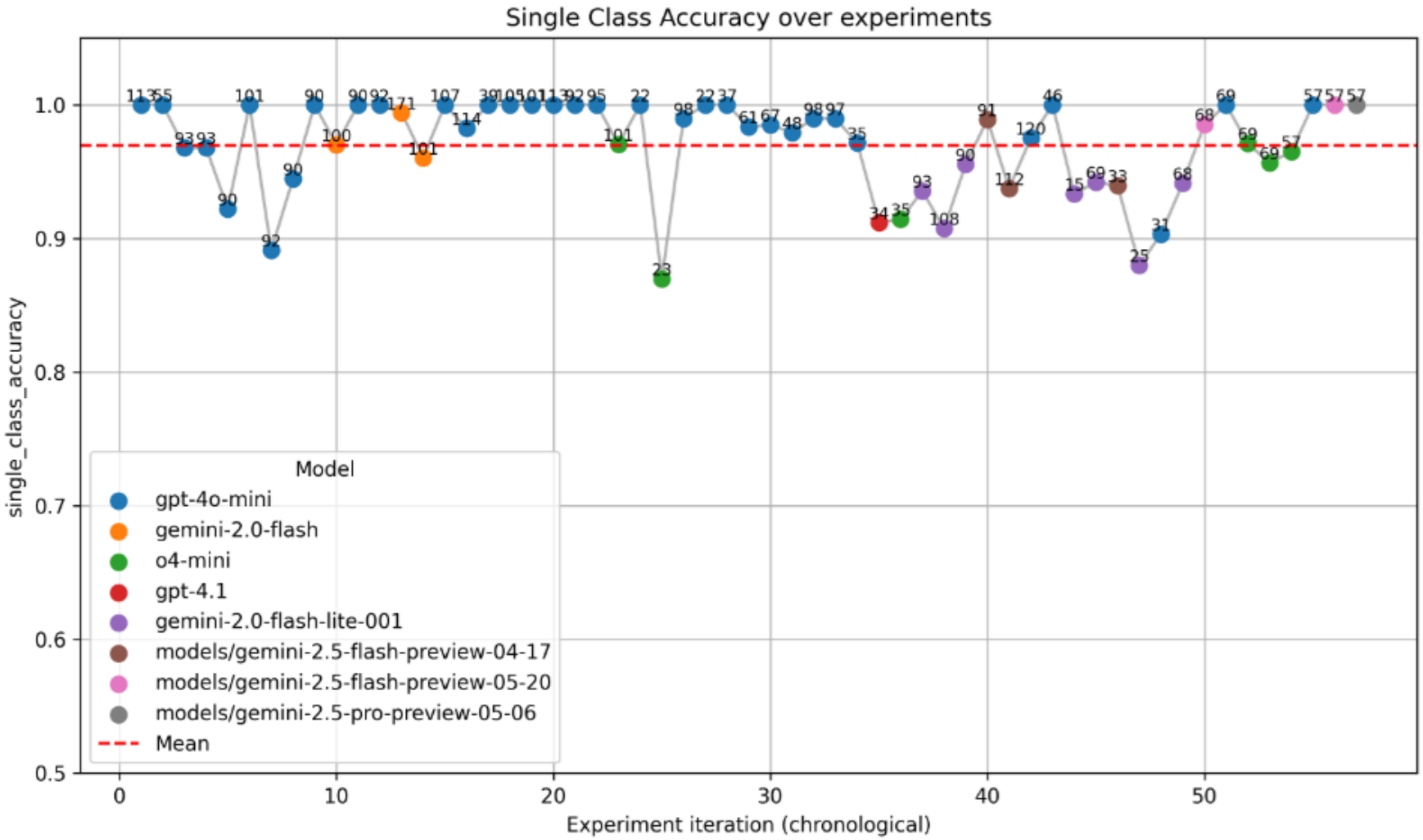
Generated Summary

```
{
  "verdict": "Malicious",
  "summary": "The code is designed to
      bypass Java's built-in
  security mechanisms, download an
      executable file
  from a specified URL, and execute it.
  This is indicative of a trojan behavior."
      ,
```

Evaluation results using different LLMs

Mean accuracy is 97%, across all 3,839 analyzed JARs and LLMs.

- 1/66 the cost of CrowdStrike Falcon
- 1/80 the cost of ANY.RUN



Favia: Forensic Agent for Vulnerability-fix Identification and Analysis

André Storhaug, Jiamou Sun, Jingyue Li,
https://arxiv.org/abs/2602.12500

Research question

Given a **CVE** (Common Vulnerabilities and Exposures) entry, identify the corresponding patch commit so others can apply a similar fix.

CVE

CVE Details: Includes the CVE ID, source identifier, publication and last modification dates, and current vulnerability status.

Known Exploited Status: Indicates whether the vulnerability is known to be actively exploited in the wild.

Scores: Severity metrics such as CVSS (Common Vulnerability Scoring System) scores, which quantify the impact and exploitability of the vulnerability.

Description: A natural language summary of the vulnerability, filtered by language preference (e.g., English).

Challenges

01

Large search space: all commits in a repository are potential candidates, making the search space prohibitively large.

- For instance, as of January 2006, the Linux kernel contains over 1.4 million commits. Only 9981 CVEs are reported in the Linux-CVE mailing list.

02

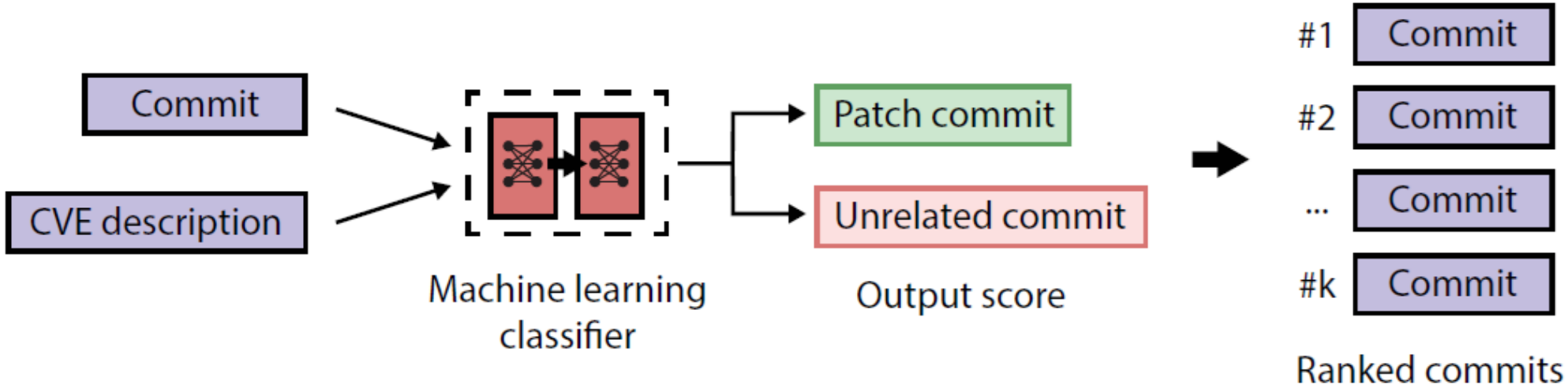
False negative: The fixes may span multiple files, involve indirect control paths, or rely on subtle behavioral changes that are not immediately evident in the diff.

03

False positive: Two patches could be similar. However, one is a valid patch, and another could be similar but does not resolve the reported vulnerability.

The approach

- **Two steps**
  - Step 1 (**Coarse narrow down**): applies a fast, coarse-grained ranking over all commits in code repository to surface a small set of candidate patches
  - Step 2 (**Detailed filtering**): Each candidate is then examined in depth by an LLM agent that performs detailed semantic reasoning over the codebase and vulnerability context to determine whether the commit constitutes a true vulnerability fix

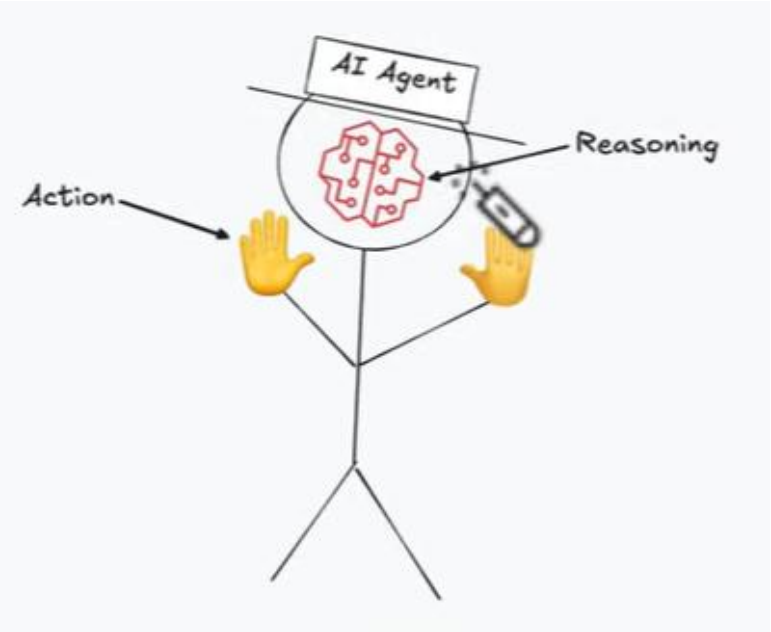


Top-n ranking using a machine learning classifier (in our study n = 10)

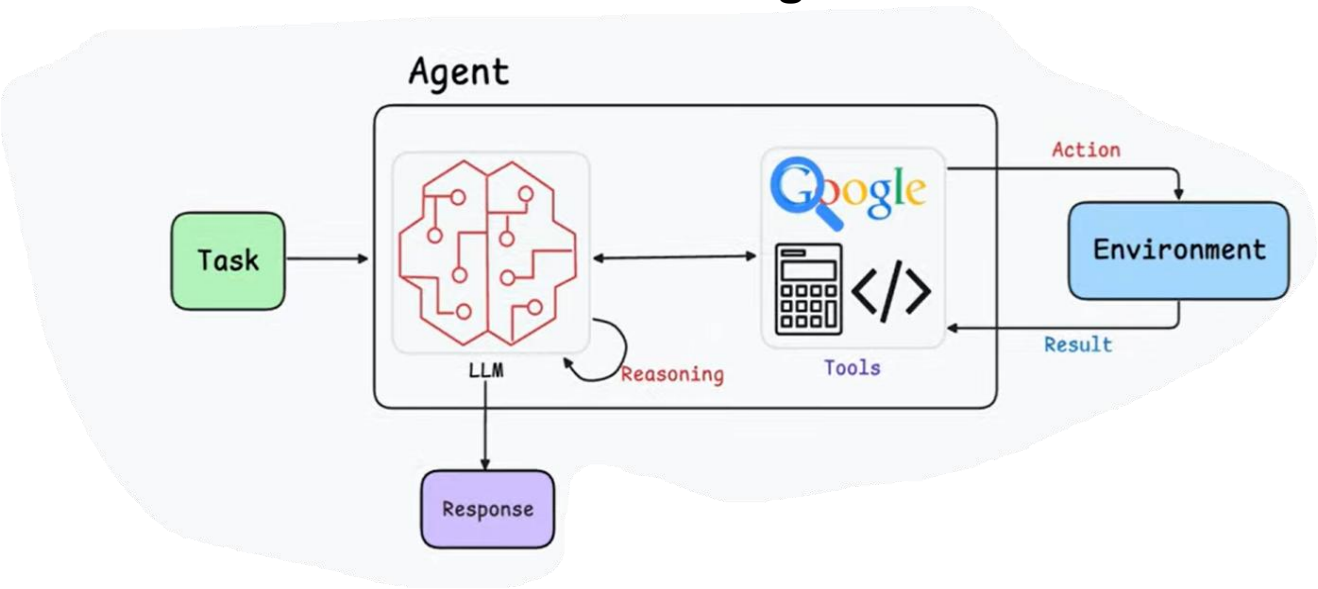
The approach – Step 1

*https://www.youtube.com/watch?v=vFdIrZyKEwQ

Agent

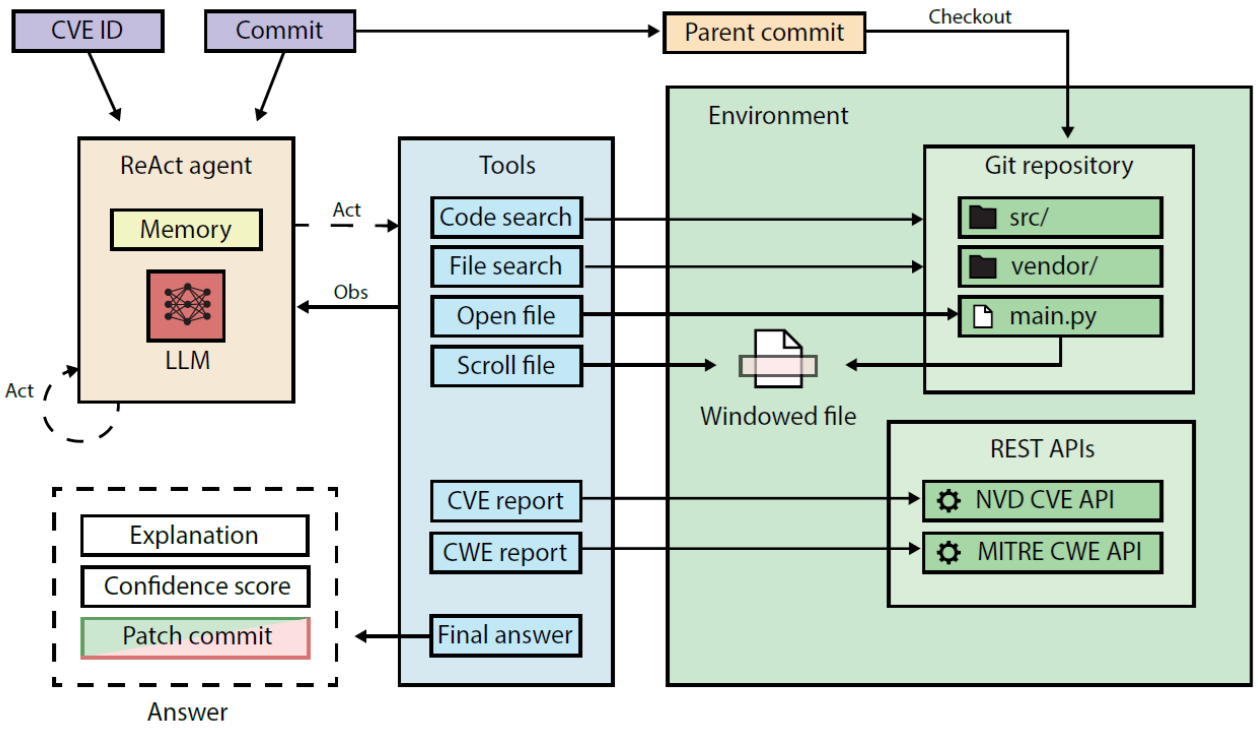


ReAct Agent *



The ReAct agent creates a feedback loop in which the model solves a problem by iteratively repeating an interleaved *thought-action-observation* process.

The approach – Step 2: ReAct Agent



ReAct Agent to classify path commit for each top-n ranked commits

Our ReAct Agent

Evaluation results

Dataset

17,293 CVEs

8,283,424 commits (23,303 patching commits)

Measurements

Precision

Recall

F-Score

Cost

Accuracy

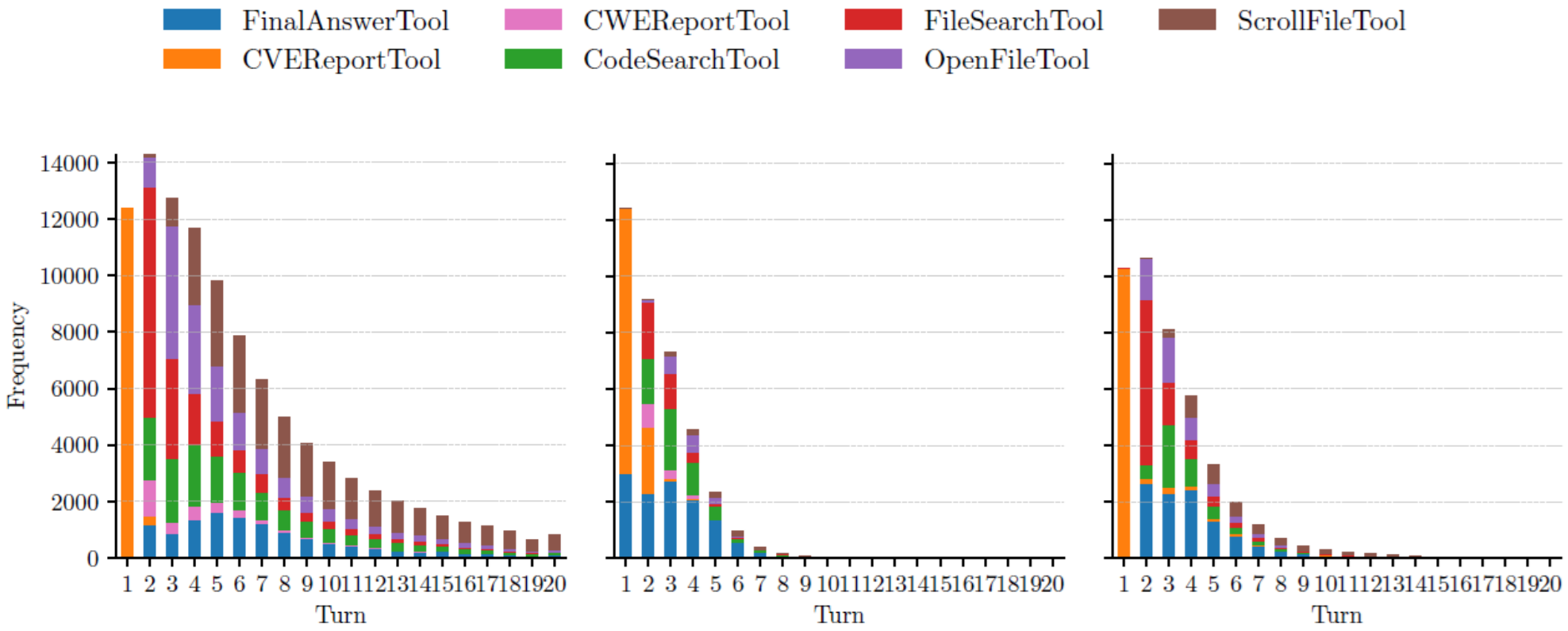
Agent-based

Classical ML-based

LLM-based

Model	Approach	Precision	Recall	F1-score
CodeBERT	VulFixMiner	0.43	0.03	0.06
CodeReviewer	PatchFinder	0.40	0.37	0.38
Llama-3.3-70B-Instruct	LLM4VFD	**0.30**	0.89	0.45
	CommitShield	0.18	0.87	0.30
	Favia	**0.30**	**0.94**	**0.46**
Qwen3-235B-A22B-Instruct-2507	LLM4VFD	0.29	0.93	0.45
	CommitShield	0.22	0.85	0.35
	Favia	**0.39**	**0.98**	**0.56**
gemma-3-27b-it	LLM4VFD	0.18	0.94	0.30
	CommitShield	0.12	**0.99**	0.21
	Favia	**0.23**	0.98	**0.37**

Bold: best-performing training approach per model.

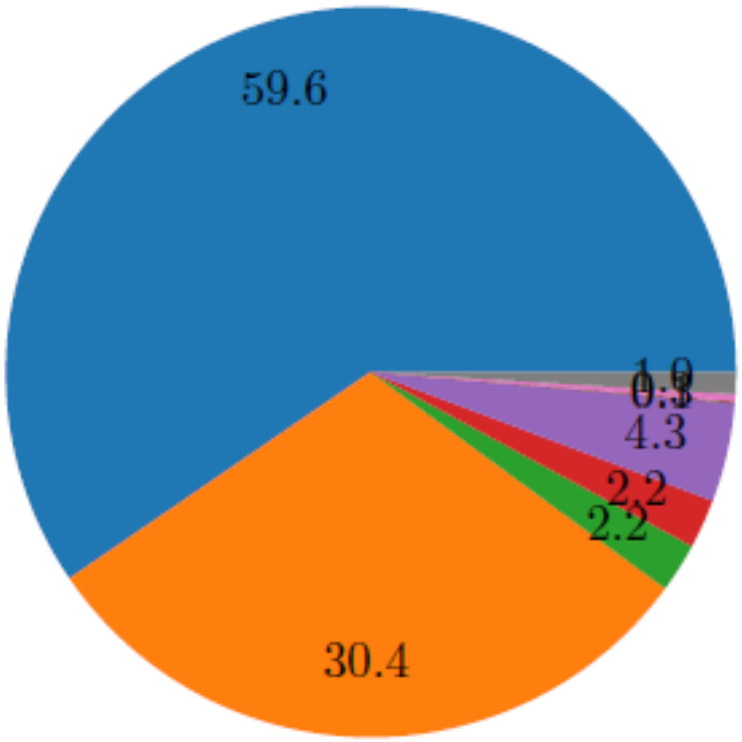


(a) Qwen3-235B-A22B-Instruct-2507 (b) Llama-3.3-70B-Instruct (c) gemma-3-27b-it

Trajectory analysis

False positive analysis

Prompt another LLM (gpt-oss-120b) to classify Favia's agent traces



Reasons for false positives

- Superficial Association
- CVE Misinterpretation
- Failed to Find Relevant Context
- Memorized CVE
- Incorrect Classification
- Ran Out Of Budget
- Gave Up Prematurely
- Other

Cost analysis

LLM-based

Mean token usage across all models

Method	Input tokens	Output tokens	Embedding tokens	Total tokens
LLM4VFD	6,456	676	362	7,494
CommitShield	9,330	1,035	0	10,365
Favia	66,159	1,043	0	67,202

Agent-based

Favia is around 1.30$ per CVE

Your experience in using AI-assisted vulnerability detection

Mentimeter

Summary

- AI-assisted software development
  - Benefits
  - Risks
- AI-assisted malware detection
- Agent-based vulnerability fixing recommendation
- Your experience in using AI-assisted vulnerability detection